Title: Programming GPIO on the CM3
Author: Craig Duffy 6/6/14, 11/01/18, 26/01/22
Module: Mobile and Embedded Devices, Secure Embedded Systems
Awards: BSc CSI, BSc Forensic Computing, Computer Security.
Prerequisites: Basic computer architecture and some C

This worksheet introduces some basic programming techniques for the Olimex boards. A very basic 'device driver' is introduced. The use of the standard peripheral library is explored.

You should clone the git repo for this worksheet https://gitlab.uwe.ac.uk/c-duffy/ses-worksheet-3.git. Once you make changes make sure that you push it to your git account and give both myself and Nathan permission to access it.

Device drivers

Once you start getting interested in computers and programming you will start to hear a lot about device drivers. They are bit of code and configuration information for controlling a device. Generally, they are associated with an operating system, such as Windows 8 or Linux. They differ from the core operating system code as they provide the interface to the myriad of differing devices that the computer may connect to. So the device drivers will change over time, as new devices come and go, and they won't be in every instance of the operating system – there is no point in having the code if you don't have the device.

In this worksheet we are going to write a couple of very simple device drivers – the first simply has 1 bit output capabilities, it flashes an LED, the second is 1 bit input, it monitors a button. Strangely there are even simpler device drivers than these, ones which, although useful, essentially do nothing!

We will use the code directory from worksheet 2. You will need to delete the main.c and create a new main.c with the code in the appendix. Also there will need to be some changes to the Makefile, specified throughout this worksheet.

In order to set up our device drivers we need a few conditions, we need to set up some time source (clocks), initialise the resource necessary for the device, and have some routines to manage the device (flash the LED, read the buttons). This is the same pattern for more complicated devices, although they are likely to have a more sophisticated resource set up and most require a de-initialisation phase to release their resources.

Clocks

The use of clocks and timers is central to computers and device control. Without a sense of time knowing what order events occurred is impossible, setting up alarms and time limits would be impossible. For example, a network device driver would have no way of knowing whether a data packet was delayed unless it had some idea of the passage of time.

On the Cortex M3 there are a number of time sources. There are internal high and low speed clock sources – HSI and LSI, as well as external high and low speed sources – HSE and LSE. We have already used and initialised these clocks in our first example piece of code without realising it. When we linked our **main.c** with the file s**tartup_stm32f10x.c** we called a routine called **SystemInit()**. This routine is part of CMSIS and is found in **STM32F10x_StdPeriph_Lib_V3.5.0/Libraries/CMSIS/CM3/DeviceSupport/ST/STM32F10x** in the file s**ystem_stm32f10x.c**. This routine is responsible for setting up the system's clocks and setting the all-important SYSTICK value. This value can be set in a range of values from 24MHz up in steps

of 12MHz to a maximum of 72MHz – dependent upon the hardware resources that have been used on the board. The Olimex board runs at 72MHz.

The CM3 has been designed to save power as much as possible and each of the separate peripheral buses can be separately clocked and initialised. There are 3 peripheral buses on the SMT32 – APB1, APB2 and AHB. The APB buses deal with peripherals – **A**dvance **P**eripheral **B**us, the AHB – **Advanced High** Speed **B**us – deals with memory devices and DMA. The GPIO pins we are dealing with are connected via the APB2 bus and the standard library gives us commands to enable those specific peripherals on that bus.

```
RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE);
```

There are **RCC** clock commands to enable and disable all of the buses and their peripherals – **RCC** stands for **R**eset and **C**lock **C**ontrol. In order to use these library features we need to include the library definitions and code – stm32f10x_rcc.[c/h]. So in the main.c file we need the include:

```
#include <stm32f10x_rcc.h>
```

and we need to alter the Makefile to build the RCC library files.

```
OBJS= $(STARTUP) stm32f10x_rcc.o stm32f10x_gpio.o main.o
```

The **bold** object files are the ones to be included – notice that **stm32f10x_gpio.o** is also included, this is being used to deal with the pins for our devices – the General Purpose Input Output pins.

<u>GPIO</u>

The STM3 has 80 GPIO pins that can be used for a variety of purposes. Sometimes they will be used to support a specific peripheral, for example a serial port or Network Interface Chip. So in the case of a NIC, it may require a pin to detect whether the cable is plugged in, and one of the GPIO pins can be used for this, but in order to do so it would need to be configured to the specific requirements in the NIC device driver code. Our driver is much simpler in that our device is a simple LED which is, of course, an output device. Firstly, we need to know which pins the LEDs on the board are connected to. Using the Olimex STM32P107 manual we can find out what we need to know – you can either find them in the schematic or in the section Input/Output. From this we can find out that Pin 63 is connected to the green LED.

The GPIO pins are organised into a series of ports on the Cortex and each port has 16 pins. The banks are labelled GPIOA, GPIOB and so on up to GPIOE. So to find a specific pin you need to know which port it is in and which pin in that port. The green LED is the 6th pin in port C – and is referred to as PC6. The standard library code has a series of routines which allow easy configuration of the various ports and their pins, this is stm32f10x_gpio.[c/h]. This will require including in the code:

```
#include <stm32f10x_gpio.h>
```

which will give us access to all the addresses of the pins and lots of settings that can be applied to them. Each port has a series of registers to control/configure the port, to read and write data to and from the pins. Access to the registers is made easier by the creation of a series of C structures that allow high level language access to them. For example in stm32f10x.h the struct GPIO_TypeDef is created.

```
typedef struct
{
```

```
  __IO uint32_t CRL;
  __IO uint32_t CRH;
  __IO uint32_t IDR;
  __IO uint32_t ODR;
  __IO uint32_t BSRR;
  __IO uint32_t BRR;
  __IO uint32_t LCKR;
} GPIO_TypeDef;
```

This allows us to access all of the registers for each port – the control registers CRL/CRH; 16-bit access data read write registers IDR/ODR, individual bit set and read registers BSRR and BRR and, finally a lock register to allow controlled individual access to the registers.

In the library there are a number of routines to allow us to initialise, read/write and lock the ports.

```
off thevoid GPIO_Init(GPIO_TypeDef* GPIOx ,
GPIO_InitTypeDef* GPIO_InitStruct);
uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx ,
uint16_t GPIO_Pin);
uint16_t GPIO_ReadInputData(GPIO_TypeDef* GPIOx);
uint8_t GPIO_ReadOutputDataBit(GPIO_TypeDef* GPIOx ,
uint16_t GPIO_Pin);
uint16_t GPIO_ReadOutputData(GPIO_TypeDef* GPIOx);
void GPIO_SetBits(GPIO_TypeDef* GPIOx , uint16_t GPIO_Pin);
void GPIO_ResetBits(GPIO_TypeDef* GPIOx , uint16_t GPIO_Pin);
void GPIO_WriteBit(GPIO_TypeDef* GPIOx , uint16_t GPIO_Pin ,
BitAction BitVal);
void GPIO_Write(GPIO_TypeDef* GPIOx , uint16_t PortVal);
```

GPIO port routines

One of the first routines we will use is GPIO_Init(), but we will need to make sure that it knows which pins we wish to configure and exactly how we intend to configure them.

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6 ;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
GPIO_Init(GPIOC, &GPIO_InitStructure);
```

This shows that we are configuring pin6 (or green LED) we are giving it a clock speed on the bus, and a mode – Output Push Pull. We then call GPIO_Init().

Once the port has been configured we can use the GPIO_WriteBit() function to toggle the LED to turn it on and off. We have to put a delay into the program otherwise it would happen so fast you wouldn't notice it.

```
GPIO_WriteBit(GPIOC , GPIO_Pin_6 , (greenledval) ? Bit_SET : Bit_RESET);
```

The code (greenledval) ? Bit_SET : Bit_RESET flips between 1 and 0 to send the correct value to the GPIO_WriteBit function. We need to toggle the value greenledval each time with:

```
greenledval = 1 – greenledval;
```

Note that for this code to work the optimisation level of the code needs to be changed in the Makefile so that the CFLAGS are like this:

```
CFLAGS  = -O0 -g
```

When you load your program onto the board use:

>flash write_image erase demo.elf 0

Note in this example we used **flash write_image erase demo.elf 0** rather than **flash write_bank** as in the previous worksheet. The **write_image** command sets up the board in order to execute the code written and initialises the board accordingly, **write_bank** can be used if we are loading data or if we don't want the processor initialised to execute the code, for example when we are going to use a debugger to set up the processor. The **write_image** command also allows us to include the **erase** parameter meaning that we don't have to do a separate erase.

Exercises.

Pass exercise 1. Write a simple program to flash the green LED in an infinite loop, put in a delay loop to stop the flashing being so fast that you won't notice it! Download and run the program.

Pass exercise 2. The following code can be used to replace the GPIO_WriteBit() function:

```
/* Set PC6  turn on*/
GPIOC->BSRR = 1 << 6;
...wait
/* reset PC6  turn off*/
GPIOC->BRR  = 1 << 6;
```

Test out that this is true by replacing the calls with this code. Using the STM32 programmers manual explain to the lab tutor how this is working. Hint read the section on GPIO. Build the library and non-library version and compare how large they are using arm-none-eabi-size command. Why are they different?

Pass exercise 3. The yellow LED is PC7. Build a new application that alternately flashes the green and yellow LEDs.

Getting Input

Getting input from the board is, as we shall see, much more complicated despite only being a 1-bit input. The Olimex board has two switches that can be programmed, there is a third button but that is a dedicated one used for resetting the system. The two user available buttons are called **WKUP** (for wakeup) and **TAMPER**, the reset button is called **RESET**. The **WKUP** button is connected to pin 23 on the board which translates to Port A pin 0 (PA0), **TAMPER** is pin 7 which translates as Port C pin 13 (PC13). First I will give details of how to program the use of the buttons.

The buttons are, like the LEDs, connected to GPIO pins, and like the LEDs the system needs programming to know how to deal with them. As with the LEDs we can use the ST Micro standard library, its data structures and functions to program the switches. We can call GPIO_Init() with the following modifications to the GPIO_InitStructure.

```
GPIO_InitStructure.GPIO_Pin =  GPIO_Pin_0;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
```

```
GPIO_Init( GPIOA, &GPIO_InitStructure );

GPIO_InitStructure.GPIO_Pin =  GPIO_Pin_13;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init( GPIOC, &GPIO_InitStructure );
```

The mode setting tells the system what type of input is expected.

The buttons can be accessed using the same library functions as were used for the LEDs, but obviously the buttons are read and the LEDs written to. So the function to read the pin/button is:

```
GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_0)
```

This will return the state of pin 0 or the **WKUP** button – 1 means pressed 0 means released.

Exercises.

Pass exercise 1. Write a simple program that reads the **WKUP** button and turns on and off the green LED each time it is pressed.

Note in order to toggle the LED between on and off you will need a variable – GreenOnOff that goes between 0 and 1. You can use a piece of code like:

```
GPIO_WriteBit(GPIOC , GPIO_Pin_6 , GreenOn);
GPIO_WriteBit(GPIOC , GPIO_Pin_7 , YellowOn);
if (GreenOn == 0 )
   GreenOn = 1;
else
   GreenOn = 0;
```

C has an interesting operator – the ?: operator that is very useful in these circumstances. This allows very efficient single line if else statements. It works by:

```
(expression1) ? (expression2) : (expression3)
```

So expression1 is evaluated and if true the result is expression2 otherwise it is expression3. So for our example above we could have:

```
GreenOn = (GreenOn == 0) ? 1 : 0 ;
```

Another way of doing this would be to use ?: in the WriteBit call

```
GPIO_WriteBit(GPIOC , GPIO_Pin_6 , (greenledval) ? Bit_SET : Bit_RESET);
greenledval = 1 - greenledval;
```

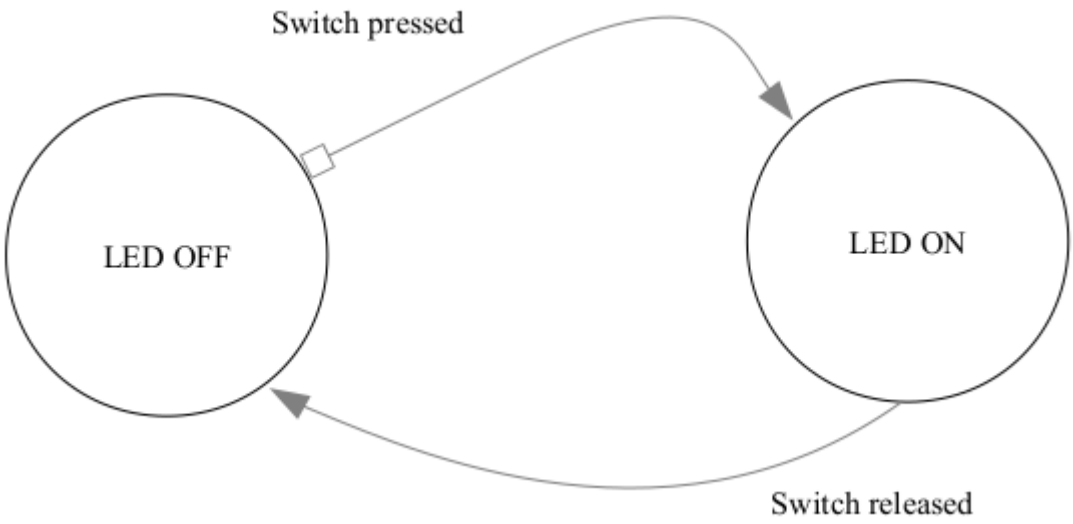
Experiment with these in your code.

Pass exercise 2. Now write a program which toggles between the yellow and green LEDs when a button is pressed.

<u>De-bouncing switch input</u>

You will notice that your switch/LED program works somewhat erratically. Sometimes it will miss switch presses other times it will respond well. This is due to 2 things. Firstly, the CPU, even on a slowish board such as this, is running at a fast rate and our fingers are slow, so this mismatch can produce odd results. Secondly, the CPU is working in a digital world where things are either 1 or 0, the switch is a mechanical device which has physical properties which are hard to represent digitally. Finally, the switches will degenerate as they get older and battered by more and more frustrated students.

Our code so far doesn't take this into account. Represented as a finite state machine it is as follows:



This translates into the following code:

```
while (1)
{
  but2_state = GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_0);
  if ( but2_state == 1 )
    GPIO_WriteBit(GPIOC , GPIO_Pin_7 ,  Bit_RESET);
  else
    GPIO_WriteBit(GPIOC , GPIO_Pin_7 ,  Bit_SET);
}
```

As we have seen this code doesn't really work. What we need to do is to de-bounce the input, this means that we need to write in an extra stage which filters the input to check to see if we have confidence that it is really a key press/release. This will require an extra state both for the key pressed and the key released using a variable that will express our confidence that the trigger event (key press/release) has really happened. The size of this variable will depend upon:

a) the reliability of the switches (pretty good in our case) and
b) how reliable we wish the code to be.

In general it is best to try various values to see their effectiveness. You might also be surprised how they can differ from board to board.

Credit exercise. Design a finite state machine and code to de-bounce the switch input and demonstrate it to your lab tutor.

Appendix A **Blink green LED code**

```
/**
  ******************************************************************************
  * @file    GPIO/IOToggle/main.c
  * @author  MCD Application Team
  * @version V3.5.0
  * @date    08-April-2011
  * @brief   Main program body.
  ******************************************************************************
  * @attention
  *
  * THE PRESENT FIRMWARE WHICH IS FOR GUIDANCE ONLY AIMS AT PROVIDING CUSTOMERS
  * WITH CODING INFORMATION REGARDING THEIR PRODUCTS IN ORDER FOR THEM TO SAVE
  * TIME. AS A RESULT, STMICROELECTRONICS SHALL NOT BE HELD LIABLE FOR ANY
  * DIRECT, INDIRECT OR CONSEQUENTIAL DAMAGES WITH RESPECT TO ANY CLAIMS ARISING
  * FROM THE CONTENT OF SUCH FIRMWARE AND/OR THE USE MADE BY CUSTOMERS OF THE
  * CODING INFORMATION CONTAINED HEREIN IN CONNECTION WITH THEIR PRODUCTS.
  *
  * <h2><center>&copy; COPYRIGHT 2011 STMicroelectronics</center></h2>
  ******************************************************************************
  */

InfoPoint@uwe.ac.uk

/* Includes ------------------------------------------------------------------*/
#include <stm32f10x.h>
#include <stm32f10x_rcc.h>
#include <stm32f10x_gpio.h>

/* Private variables ---------------------------------------------------------*/
GPIO_InitTypeDef GPIO_InitStructure;

int main(void)
{
  volatile uint32_t i;
  static int greenledval = 0;


  /*!< At this stage the microcontroller clock setting is already configured,
       this is done through SystemInit() function which is called from startup
       file (startup_stm32f10x_xx.s) before to branch to application main.
       To reconfigure the default setting of SystemInit() function, refer to
       system_stm32f10x.c file
     */

  /* GPIOC Periph clock enable */
  RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE);

  /* Configure PC6 in output pushpull mode */
  GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6 ;
  GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
  GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
  GPIO_Init(GPIOC, &GPIO_InitStructure);


  /* To achieve GPIO toggling maximum frequency, the following sequence is
     mandatory.
     You can monitor PD0 or PD2 on the scope to measure the output signal.
     If you need to fine tune this frequency, you can add more GPIO set/reset
     cycles to minimize more the infinite loop timing.
     This code needs to be compiled with high speed optimization option.  */

  while (1)
```

```
  {
    GPIO_WriteBit(GPIOC , GPIO_Pin_6 , (greenledval) ? Bit_SET : Bit_RESET);
    greenledval = 1 - greenledval;

    i = 1000000;     /* Delay */
    while(--i);

    GPIO_WriteBit(GPIOC , GPIO_Pin_6 , (greenledval) ? Bit_SET : Bit_RESET);
    greenledval = 1 - greenledval;

    i = 8000000;     /* Delay */
    while(--i);
  }
}

#ifdef  USE_FULL_ASSERT

/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */

void assert_failed(uint8_t* file, uint32_t line)
{
  /* User can add his own implementation to report the file name and
     line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */

  /* Infinite loop */
  while (1)
  {
  }
}

#endif
/****************** (C) COPYRIGHT 2011 STMicroelectronics *****END OF FILE****/
```

Appendix B Note about tar files

If you are not or unable to use git then follow the instructions given below.

This worksheet requires you to download the worksheet3.tgz file and untar it. How to do this is explained in the previous worksheet. You should make sure that you untar this directory into the same directory that you untarred your previous tar file. The makefile in the directory is expecting the ST Micro peripheral library to be in a series of sub-directories in the parent directory of the current one.